

MODULE 2

Verrouiller l'authentification

Module 2. Verrouiller l'authentification

Ce module audite et corrige toutes les failles d'authentification de TaskFlow : mots de passe stockés en clair, énumération de comptes, brute force illimité, politique de mot de passe absente. À la fin, la route `/auth` est sécurisée, les comptes existants migrés et les tests de validation passent.

Objectifs opérationnels

À la fin de ce module, tu auras dans `vuln-app/` :

- ☒ Mots de passe hashés avec bcrypt (coût 12) dans la table `users`
- ☒ Messages d'erreur génériques (`"Identifiants invalides"`) dans `/auth/login`
- ☒ Rate limiting actif sur `/auth/login` : 5 tentatives / 15 min
- ☒ Validation de politique de mot de passe (12 caractères minimum)
- ☒ Aucun mot de passe visible dans les logs serveur
- ☒ Script de migration `scripts/migrate-passwords.js` commité
- ☒ Commit tagué `M2-auth-securise`

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
1	Audit du code existant	Lire <code>auth.js</code> + identifier les 4 failles	10 min
2	Démonstration exploit	Brute force + énumération de comptes	10 min
3	Hachage bcrypt	Installer bcryptjs + corriger register + migration	20 min
4	Messages génériques	Correction login + test curl	5 min
5	Rate limiting	Installer express-rate-limit + configurer	10 min
6	Tests de validation	6 commandes curl de vérification	10 min

CONCEPT 1, Hachage vs chiffrement

Rappel. Le **chiffrement** est réversible : avec la clé, on récupère la donnée originale (AES, RSA). Il sert à protéger des données qu'on doit relire (messages, fichiers). Le **hachage** est irréversible : on ne peut pas retrouver le mot de passe original à partir du hash. Il sert à vérifier sans stocker. Pour les mots de passe, on hash toujours, on ne chiffre jamais. Si un attaquant vole la base, il obtient des hashes impossibles à inverser directement, pas des mots de passe exploitables.

Risque concret	Si la base SQLite est volée (injection SQL, path traversal, backup mal protégé), les mots de passe en clair sont immédiatement exploitables sur tous les services de l'utilisateur.
Exemple de faille	Voir <code>server/routes/auth.js</code> ligne 24, insertion en clair
À vérifier dans l'app	Ouvrir la DB : <code>sqlite3 db/taskflow.db</code> <code>"SELECT email, password FROM users LIMIT 3"</code>
Correction	Remplacer l'insertion en clair par <code>bcrypt.hash(password, 12)</code>
Test de validation	Après correction, <code>SELECT password FROM users</code> retourne des chaînes commençant par <code>\$2b\$</code>
Trace rapport	“La faille M2-1 (mots de passe en clair) a été corrigée. Après correction, la colonne <code>users.password</code> ne contient que des hashes <code>bcrypt</code> (coût 12). Vérification : <code>SELECT password FROM users LIMIT 1</code> retourne <code>\$2b\$12\$... .</code> ”

Code vulnérable (avant) :

```
// server/routes/auth.js, AVANT (vulnérable)
router.post('/register', (req, res) => {
  const { email, password } = req.body;
  // ...
  const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES (?, ?, ?)');
  stmt.run(email, password, 'user'); // VULN: password en clair
  res.json({ message: 'User created' });
});
```

Code corrigé (après) :

```
// server/routes/auth.js, APRÈS (sécurisé)
const bcrypt = require('bcryptjs');

router.post('/register', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'email and password required' });
  }

  // Politique de mot de passe : 12 chars minimum
  if (password.length < 12) {
    return res.status(400).json({ error: 'Password must be at least 12 characters'
});
  }

  try {
    const hash = await bcrypt.hash(password, 12); // coût 12
    const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES (?,
?, ?)');
    stmt.run(email, hash, 'user');
    res.status(201).json({ message: 'User created' });
  } catch (err) {
    // Pas de fuite de stack trace
    res.status(500).json({ error: 'Registration failed' });
  }
});
```

CONCEPT 2, Sel (salt)

Rappel. Un sel est une valeur aléatoire unique ajoutée à chaque mot de passe avant le hachage. Sans sel, deux utilisateurs avec le même mot de passe auraient le même hash, une attaque par table arc-en-ciel (rainbow table) préalablement calculée permettrait de casser des milliers de comptes d'un coup. Avec sel, chaque hash est unique même si les mots de passe sont identiques. **bcrypt intègre le sel automatiquement** dans le hash produit, il n'y a rien à gérer manuellement. Le hash stocké contient le sel, le coût et le hash : `$2b$12$[sel_22chars][hash_31chars]` .

Risque concret	Sans sel, une base de 1000 comptes avec <code>"password123"</code> produit 1000 fois le même hash. Une seule entrée dans la rainbow table compromet 1000 comptes.
Exemple de faille	<code>sha256("password123")</code> produit toujours <code>ef92b778 ...</code> , prévisible et pré-calculable
À vérifier dans l'app	Après migration, chaque ligne de <code>users.password</code> doit commencer par <code>\$2b\$12\$</code> et être unique
Correction	<code>bcrypt</code> gère le sel automatiquement, utiliser <code>bcrypt.hash(pwd, 12)</code> suffit
Test de validation	<code>SELECT password FROM users WHERE role='user'</code> : les 2 hashes doivent être différents même si <code>alice123</code> et <code>bob456</code> ont des longueurs similaires
Trace rapport	“Les hashes <code>bcrypt</code> stockés intègrent un sel unique par compte (visible dans les 22 premiers caractères après <code>\$2b\$12\$</code>). Une attaque par rainbow table est inefficace car chaque hash est unique.”

Illustration du format `bcrypt` :

```
$2b$12$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
| | |-----|-----|
| | sel aléatoire (22 chars) hash du mot de passe (31 chars)
| coût (2^12 = 4096 itérations)
algorithme bcrypt
```

CONCEPT 3, `bcrypt` vs `argon2`

Rappel. `bcrypt` (1999) et `argon2` (2015, vainqueur Password Hashing Competition) sont tous deux conçus pour être lents et coûteux en mémoire, c'est intentionnel pour ralentir les attaques par force brute. **`bcrypt`** : paramètre de coût (`saltRounds`), large adoption, bibliothèque stable. **`argon2`** : plus récent, paramètres mémoire + temps + parallélisme, recommandé pour les nouveaux projets. Pour ce cours, on utilise **`bcryptjs`** (pure JS, pas de dépendance native) avec un coût de 12, ce qui donne ~250ms de calcul par hash, acceptable pour le login, dissuasif pour l'attaque.

Risque concret	SHA-256 sans sel calcule 10 milliards de hashes/seconde sur un GPU. bcrypt coût 12 : ~250ms/hash, rend le brute force des milliers de fois moins efficace.
Exemple de faille	MD5 ou SHA-256 pour stocker des mots de passe (encore présent dans des apps legacy)
À vérifier dans l'app	<code>package.json</code> contient-il <code>bcryptjs</code> ? Est-il utilisé dans <code>auth.js</code> avec <code>await</code> ?
Correction	<code>npm install bcryptjs</code> (déjà dans <code>package.json</code>) + utiliser <code>bcrypt.hash(pwd, 12)</code>
Test de validation	<code>node -e "const b=require('bcryptjs'); b.hash('test',12).then(h=>console.log(h.startsWith('\$2b\$12\$')))"</code> doit afficher <code>true</code>
Trace rapport	“L’algorithme de hachage bcryptjs (coût 12, ~250ms/hash) a été sélectionné. Ce coût rend une attaque par dictionnaire de 10 000 mots de passe non-parallélisable en moins de 41 minutes par thread d’attaque.”

CONCEPT 4, Politique de mot de passe

Rappel. Les recherches (NIST SP 800-63B) montrent que la **longueur** est plus efficace que la complexité forcée (majuscules + chiffres + symboles). Un mot de passe de 16 caractères simples est plus solide qu’un de 8 caractères complexes. Les règles de complexité poussent les utilisateurs à des patterns prévisibles (`Password1!`). Recommandation NIST actuelle : minimum 8 caractères (12 en entreprise), pas de rotation forcée, pas de règles de complexité absurdes, vérification contre les listes de mots de passe compromis si possible.

Risque concret	TaskFlow accepte actuellement "a" comme mot de passe valide. alice123 (7 chars) est dans les 1000 premiers mots de passe les plus courants.
Exemple de faille	<pre>POST /auth/register {"email": "x@x.com", "password": "a"} , crée un compte avec mot de passe d'1 caractère</pre>
À vérifier dans l'app	Existe-t-il une validation de longueur dans register ?
Correction	Ajouter <code>if (password.length < 12) return res.status(400).json({ ... })</code>
Test de validation	<pre>curl -X POST http://localhost:3000/auth/register -H "Content-Type: application/json" -d '{"email": "test@test.com", "password": "court"}'</pre> doit retourner 400
Trace rapport	“Une validation de politique de mot de passe a été ajoutée dans /auth/register : longueur minimum 12 caractères. Les mots de passe alice123 et bob456 des comptes de test ont été rejetés et remplacés lors de la migration.”

CONCEPT 5, Messages d'erreur génériques

Rappel. Quand un attaquant teste `POST /login` avec un email, la réponse lui apprend s'il est sur la bonne piste : `"User not found"` confirme que l'email n'existe pas, `"Wrong password"` confirme que l'email existe. Cette information permet d'**énumérer les comptes** : il suffit de tester des emails jusqu'à obtenir "Wrong password". La correction est simple : retourner le même message dans les deux cas. Retour attendu : `"Identifiants invalides"` ou `"Invalid credentials"`, quelle que soit la cause du refus.

Risque concret	En 10 minutes avec une liste d'emails courants (HaveIBeenPwned), un attaquant peut identifier tous les comptes existants dans TaskFlow.
Exemple de faille	<code>server/routes/auth.js</code> lignes 52-55 : deux messages d'erreur différents
À vérifier dans l'app	<code>grep -n "not found\ wrong\ incorrect\ invalid" server/routes/auth.js</code>
Correction	Un seul message dans tous les cas de refus + même code HTTP (401)
Test de validation	<code>curl -X POST http://localhost:3000/auth/login -d '{"email":"inconnu@x.com","password":"x"}'</code> et <code>curl -X POST http://localhost:3000/auth/login -d '{"email":"alice@taskflow.io","password":"mauvais"}'</code> doivent retourner le même corps JSON
Trace rapport	“La faille d'énumération de comptes (M2-2) a été corrigée. Les routes <code>/auth/login</code> retournent désormais <code>{"error\":"Identifiants invalides\"}</code> dans tous les cas de refus, sans différencier compte inexistant et mot de passe incorrect.”

Code vulnérable (avant) :

```
// AVANT, deux messages différents = énumération possible
if (!user) {
  return res.status(401).json({ error: 'User not found' }); // révèle que le
  compte n'existe pas
}
if (user.password !== password) {
  return res.status(401).json({ error: 'Wrong password' }); // révèle que le
  compte existe
}
```

Code corrigé (après) :

```
// APRÈS, message uniforme
const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);

if (!user) {
  return res.status(401).json({ error: 'Identifiants invalides' });
}

const passwordMatch = await bcrypt.compare(password, user.password);
if (!passwordMatch) {
  return res.status(401).json({ error: 'Identifiants invalides' });
}
```

CONCEPT 6, Rate limiting / anti-brute force

Rappel. Sans limite de taux, un attaquant peut tester des milliers de mots de passe par seconde sur `/login`. `express-rate-limit` compte les requêtes par IP (ou par compte) sur une fenêtre de temps et bloque quand le seuil est atteint. Configuration recommandée pour le login : **5 tentatives par 15 minutes par IP**. Après le blocage, répondre avec `429 Too Many Requests`. Important : le message de blocage ne doit pas confirmer que le compte existe, utiliser le même message générique.

Risque concret	Sans rate limit, <code>rockyou.txt</code> (14 millions de mots de passe) peut être testé en quelques minutes si le serveur est rapide. Même avec bcrypt, 250ms × N requêtes parallèles reste faisable.
Exemple de faille	Boucle curl ci-dessous, 10 tentatives sans aucun blocage
À vérifier dans l'app	<code>grep -r "rateLimit\ rate-limit\ express-rate" server/</code> , doit retourner vide dans la version vulnérable
Correction	<code>npm install express-rate-limit + middleware</code> sur <code>/auth/login</code>
Test de validation	6 tentatives consécutives → la 6ème retourne HTTP 429
Trace rapport	“Un rate limiter a été ajouté sur <code>/auth/login</code> : 5 tentatives par 15 minutes par adresse IP (express-rate-limit v7). Les tentatives au-delà du seuil retournent HTTP 429 avec le message <code>"Trop de tentatives, réessayez dans 15 minutes."</code> ”

Code corrigé :

```
// server/routes/auth.js, ajout en haut du fichier
const rateLimit = require('express-rate-limit');

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 tentatives max par IP
  message: { error: 'Trop de tentatives, réessayez dans 15 minutes.' },
  standardHeaders: true, // retourne les headers RateLimit-*
  legacyHeaders: false,
});

// Appliquer uniquement sur /login
router.post('/login', loginLimiter, async (req, res) => {
  // ... reste du code
});
```

CONCEPT 7, Stockage sécurisé des identifiants

Rappel. Un mot de passe ne doit jamais apparaître dans les logs, même en développement. Les logs sont souvent moins bien protégés que la base de données (rotation, accès en lecture, archivage externe). `console.log('login attempt:', email, password)` expose le mot de passe en clair dans les logs serveur. Il faut aussi vérifier que les variables d'environnement contenant des secrets ne sont pas loggées au démarrage. Règle : **ne logger que ce qui est nécessaire au débogage sans inclure de données sensibles.**

Risque concret	Les logs d'un serveur Node.js sont souvent lisibles par tous les processus du même serveur. Un attaquant avec accès SSH limité peut lire les tentatives de login avec mots de passe.
Exemple de faille	<code>server/routes/auth.js</code> ligne 44 : <code>console.log('login attempt:', email, password)</code>
À vérifier dans l'app	<code>grep -rn "console.log" server/ \ grep -i "password\ pwd\ secret\ token"</code>
Correction	Logger uniquement l'email (jamais le mot de passe) : <code>console.log('login attempt:', email)</code>
Test de validation	Déclencher un login depuis le terminal, vérifier les logs serveur, aucune valeur de password ne doit apparaître
Trace rapport	“La faille M6-1 (log du mot de passe en clair) a été corrigée. La ligne <code>console.log('login attempt:', email, password)</code> a été remplacée par <code>console.log('login attempt:', email)</code> . Vérification : <code>grep -n 'password' server/routes/auth.js</code> ne retourne aucune ligne de log.”

Audit rapide des logs sensibles :

```
# Vérifier tous les console.log potentiellement dangereux
grep -rn "console\.log" vuln-app/server/ | grep -iE "password|pwd|secret|token|key"
```

TP complet, Étapes séquentielles

Étape 1, Audit du code auth actuel (10 min)

```
# Lire le fichier d'authentification
cat vuln-app/server/routes/auth.js

# Vérifier les mots de passe dans la DB
cd vuln-app
node -e "
  const db = require('./server/db');
  const users = db.prepare('SELECT id, email, password FROM users').all();
  users.forEach(u => console.log(u.id, u.email, '|', u.password));
"
```

Ce que tu dois voir :

```
1 alice@taskflow.io | alicel23      ← mot de passe en clair
2 bob@taskflow.io | bob456          ← mot de passe en clair
3 admin@taskflow.io | adminpass     ← mot de passe en clair
```

Liste des failles à corriger :

#	FAILLE	LIGNE DANS AUTH.JS	MODULE
F1	Mot de passe stocké en clair	~24 (INSERT)	M2
F2	Comparaison en clair au lieu de bcrypt.compare	~52	M2
F3	Message “User not found” vs “Wrong password”	~49, ~54	M2
F4	Pas de rate limit sur /login	,	M2
F5	Log du mot de passe	~44	M2
F6	Pas de politique de mot de passe	,	M2

Étape 2, Démonstration de l'exploit (10 min)

⚠ Cette étape montre pourquoi les corrections sont nécessaires. Ne jamais utiliser ces techniques sur une app non autorisée.

2a, Énumération de comptes (avant correction) :

```
# Test avec un email inexistant → "User not found"
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"inconnu@exemple.com","password":"test"}' | jq .

# Test avec un email existant → "Wrong password"
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@taskflow.io","password":"test"}' | jq .
```

Résultat attendu (vulnérable) :

```
{"error": "User not found"}    ← l'email n'existe pas
{"error": "Wrong password"}   ← l'email EXISTE, information précieuse
```

2b, Brute force (avant correction) :

```
# Simuler un brute force simple (10 tentatives sans blocage)
for i in $(seq 1 10); do
  curl -s -X POST http://localhost:3000/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email":"alice@taskflow.io","password":"tentative$i"}'
  echo ""
done
# Toutes les 10 requêtes passent, pas de blocage
```

Résultat attendu (vulnérable) : 10 réponses `{"error": "Wrong password"}` , jamais de 429.

Étape 3, Correction bcrypt + migration (20 min)

3a, Installer les dépendances :

```
cd vuln-app
# bcryptjs est déjà dans package.json, vérifier :
cat package.json | grep bcryptjs
# Si absent : npm install bcryptjs
# express-rate-limit à ajouter :
npm install express-rate-limit
```

3b, Réécrire `server/routes/auth.js` :

```
/**
 * auth.js, Routes d'authentification (VERSION CORRIGÉE M2)
 *
 * Corrections apportées :
 * M2-1 : bcrypt.hash() dans /register, bcrypt.compare() dans /login
 * M2-2 : messages d'erreur uniformes ("Identifiants invalides")
 * M2-3 : rate limiting 5 req / 15 min sur /login
 * M2-4 : politique mdp 12 chars minimum
 * M6-1 : suppression log du mot de passe
 */

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const rateLimit = require('express-rate-limit');
const db = require('../db');

const router = express.Router();

const JWT_SECRET = process.env.JWT_SECRET || 'CHANGE_ME_IN_PRODUCTION';

// Rate limiter : 5 tentatives de login par 15 minutes par IP
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Trop de tentatives, réessayez dans 15 minutes.' },
  standardHeaders: true,
  legacyHeaders: false,
});

/**
 * POST /auth/register
 */
router.post('/register', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ error: 'email and password required' });
  }

  // M2-4 : politique de mot de passe
  if (password.length < 12) {
    return res.status(400).json({ error: 'Le mot de passe doit contenir au moins 12 caractères' });
  }

  try {
    // M2-1 : hashage bcrypt coût 12
    const hash = await bcrypt.hash(password, 12);
    const stmt = db.prepare('INSERT INTO users (email, password, role) VALUES (?, ?, ?)');
  }
}
```

```

    stmt.run(email, hash, 'user');
    res.status(201).json({ message: 'Compte créé' });
  } catch (err) {
    if (err.message.includes('UNIQUE constraint')) {
      return res.status(409).json({ error: 'Cet email est déjà utilisé' });
    }
    res.status(500).json({ error: 'Erreur lors de la création du compte' });
  }
});

/**
 * POST /auth/login
 */
router.post('/login', loginLimiter, async (req, res) => {
  const { email, password } = req.body;

  // M6-1 : ne loguer QUE l'email, jamais le mot de passe
  console.log('login attempt:', email);

  if (!email || !password) {
    return res.status(400).json({ error: 'email and password required' });
  }

  try {
    const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email);

    // M2-2 : même message quel que soit le motif de refus
    if (!user) {
      return res.status(401).json({ error: 'Identifiants invalides' });
    }

    // M2-1 : comparaison via bcrypt
    const passwordMatch = await bcrypt.compare(password, user.password);
    if (!passwordMatch) {
      return res.status(401).json({ error: 'Identifiants invalides' });
    }

    const token = jwt.sign(
      { id: user.id, email: user.email, role: user.role },
      JWT_SECRET,
      { expiresIn: '2h' } // durée réduite (7d était excessif)
    );

    res.json({ token, role: user.role });
  } catch (err) {
    res.status(500).json({ error: 'Erreur lors de la connexion' });
  }
});

module.exports = router;
module.exports.JWT_SECRET = JWT_SECRET;

```


3c, Script de migration `scripts/migrate-passwords.js` :

```
mkdir -p vuln-app/scripts
```

```
// scripts/migrate-passwords.js
// Migre les mots de passe en clair vers bcrypt
// Usage : node scripts/migrate-passwords.js

const bcrypt = require('bcryptjs');
const Database = require('better-sqlite3');
const path = require('path');

const DB_PATH = path.join(__dirname, '..', 'db', 'taskflow.db');
const db = new Database(DB_PATH);

const SALT_ROUNDS = 12;

async function migratePasswords() {
  const users = db.prepare('SELECT id, email, password FROM users').all();

  console.log(`Migration de ${users.length} comptes ...`);

  for (const user of users) {
    // Détecter si le mot de passe est déjà hashé (bcrypt commence par $2b$)
    if (user.password.startsWith('$2b$') || user.password.startsWith('$2a$')) {
      console.log(` ✓ ${user.email} : déjà hashé, ignoré`);
      continue;
    }

    const hash = await bcrypt.hash(user.password, SALT_ROUNDS);
    db.prepare('UPDATE users SET password = ? WHERE id = ?').run(hash, user.id);
    console.log(` ✓ ${user.email} : migré (plain → bcrypt)`);
  }

  console.log('Migration terminée.');
```

```
  // Vérification
  const sample = db.prepare('SELECT email, password FROM users LIMIT 1').get();
  console.log(`\nVérification : ${sample.email} → ${sample.password.substring(0,
20)} ...`);
  console.log(`Hash valide : ${sample.password.startsWith('$2b$')}`);
}

migratePasswords().catch(console.error);
```

Exécuter la migration :

```
cd vuln-app
node scripts/migrate-passwords.js
```

Résultat attendu :

```
Migration de 3 comptes ...
  ✓ alice@taskflow.io : migré (plain → bcrypt)
  ✓ bob@taskflow.io : migré (plain → bcrypt)
  ✓ admin@taskflow.io : migré (plain → bcrypt)
Migration terminée.

Vérification : alice@taskflow.io → $2b$12$N9qo8uL0ickgx2 ...
Hash valide : true
```

Étape 4, Vérification des messages d'erreur (5 min)

```
# Démarrer l'app si pas déjà lancée
# node vuln-app/server/index.js &

# Test 1 : email inexistant
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"inconnu@exemple.com","password":"test"}'
# Attendu : {"error":"Identifiants invalides"}

# Test 2 : email existant, mauvais mdp
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@taskflow.io","password":"mauvaismdp"}'
# Attendu : {"error":"Identifiants invalides"} ← MÊME message qu'au-dessus
```

✓ **Point de vérification** : les deux réponses ont exactement le même corps JSON.

Étape 5, Test du rate limiting (10 min)

```
# Envoyer 6 requêtes, la 6ème doit retourner 429
for i in $(seq 1 6); do
  echo -n "Tentative $i : "
  curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:3000/auth/login \
    -H "Content-Type: application/json" \
    -d "{\"email\":\"alice@taskflow.io\",\"password\":\"tentative$i\"}"
  echo ""
done
```

Résultat attendu :

```
Tentative 1 : 401
Tentative 2 : 401
Tentative 3 : 401
Tentative 4 : 401
Tentative 5 : 401
Tentative 6 : 429 ← bloqué
```

Étape 6, Tests de validation complets (10 min)

```
# Test 1 : Politique de mot de passe, mot de passe trop court
curl -s -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"court"}' | jq .
# Attendu : 400 + message longueur minimum

# Test 2 : Inscription valide avec mot de passe fort
curl -s -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@test.com","password":"motDePasseLong2024!"}' | jq .
# Attendu : 201 + {"message":"Compte créé"}

# Test 3 : Login valide (alice après migration)
# Note : alice123 ne fonctionnera plus, mot de passe trop court si re-register
# Utiliser le compte newuser créé ci-dessus :
curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@test.com","password":"motDePasseLong2024!"}' | jq .
# Attendu : {"token":"eyJ...","role":"user"}

# Test 4 : Vérifier la DB, tous les hashes commencent par $2b$
node -e "
  const db = require('./server/db');
  const users = db.prepare('SELECT email, password FROM users').all();
  users.forEach(u => {
    const ok = u.password.startsWith('\$2b\$');
    console.log(ok ? '✓' : 'X', u.email, '|', u.password.substring(0, 20) +
' ... ');
  });
"

# Test 5 : Vérifier aucun password dans les logs
grep -n "password" vuln-app/server/routes/auth.js | grep "console.log"
# Attendu : aucune ligne
```

Atelier : Stockage sécurisé des identifiants

Cet atelier complète la correction M2 en s'assurant qu'aucun chemin de code ne fuit d'identifiants.

Audit systématique :

```
# 1. Chercher tous les console.log avec données sensibles
grep -rn "console\.log\|console\.error\|console\.warn" vuln-app/server/ \
| grep -iE "password|pwd|secret|token|key|credential"

# 2. Vérifier que JWT_SECRET vient bien de l'environnement
grep -n "JWT_SECRET" vuln-app/server/routes/auth.js
# Attendu : process.env.JWT_SECRET, pas de valeur en dur

# 3. Vérifier que .env n'est pas commité
git -C vuln-app log --oneline --all -- .env
# Attendu : aucune ligne (fichier jamais commité)

# 4. Vérifier le contenu de db.js, credentials en dur (VULN M6 hors périmètre M2)
grep -n "const DB_" vuln-app/server/db.js
# Ces constantes inutilisées seront corrigées en M6
```

Résultat attendu de l'audit M2 :

CHECK	RÉSULTAT	STATUT
Aucun console.log(password)	0 ligne trouvée	✓
JWT_SECRET depuis process.env	Ligne trouvée	✓
.env non commité	Aucun commit	✓
Hash bcrypt dans tous les mots de passe	\$2b\$ pour chaque user	✓

Mini-quiz du module (8 QCM)

Q1. Quelle est la différence principale entre hachage et chiffrement ?

- A. Le hachage utilise une clé, le chiffrement non
- B. Le hachage est irréversible, le chiffrement est réversible avec la clé
- C. Le hachage est plus rapide que le chiffrement
- D. Il n'y a pas de différence pour stocker des mots de passe

Q2. À quoi sert le sel (salt) dans bcrypt ?

- A. À ralentir le calcul du hash
- B. À éviter que deux mots de passe identiques produisent le même hash

- C. À chiffrer le mot de passe avant le hachage
- D. À allonger le mot de passe automatiquement

Q3. Quel message d'erreur de login évite l'énumération de comptes ?

- A. "Utilisateur non trouvé"
- B. "Mot de passe incorrect"
- C. "Identifiants invalides"
- D. "Email ou mot de passe incorrect"

Q4. Le cost factor 12 de bcrypt signifie :

- A. 12 octets de sel
- B. 12 secondes de calcul minimum
- C. $2^{12} = 4096$ itérations de hachage
- D. Une longueur de hash de 12 caractères

Q5. Qu'est-ce qu'une rainbow table ?

- A. Une table de couleurs pour interfaces graphiques
- B. Une base de données de hashes précalculés associés à leurs mots de passe sources
- C. Un outil de gestion de sessions
- D. Une technique de salage automatique

Q6. Le rate limiting sur `/login` est configuré à 5 req / 15 min. La 6ème tentative retourne :

- A. HTTP 401 comme les précédentes
- B. HTTP 403 Forbidden
- C. HTTP 429 Too Many Requests
- D. HTTP 503 Service Unavailable

Q7. Pourquoi bcrypt est-il préféré à SHA-256 pour les mots de passe ?

- A. bcrypt produit des hashes plus courts
- B. SHA-256 n'accepte pas les caractères spéciaux
- C. bcrypt est intentionnellement lent (coût configurable), SHA-256 est trop rapide pour le brute force
- D. bcrypt est compatible avec tous les navigateurs

Q8. Dans le script de migration, pourquoi vérifier `password.startsWith('$2b$')` ?

- A. Pour vérifier que le mot de passe est assez long

- B. Pour éviter de double-hashier un mot de passe déjà migré
- C. Pour valider la politique de mot de passe
- D. C'est une syntaxe bcrypt obligatoire

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	Hachage unidirectionnel = irréversible ; chiffrement = réversible avec clé
Q2	B	Le sel garantit l'unicité des hashes même pour mots de passe identiques
Q3	C	"Identifiants invalides" ne révèle ni si l'email existe ni si c'est le mot de passe
Q4	C	cost=12 → 2 ¹² tours d'itération → ~250ms sur hardware moderne
Q5	B	Base de hashes précalculés ; le sel les rend inefficaces
Q6	C	429 Too Many Requests est le code standard pour rate limiting
Q7	C	SHA-256 : ~10 milliards hashes/s sur GPU. bcrypt coût 12 : ~250ms/hash
Q8	B	Un hash bcrypt commence toujours par \$2b\$ ou \$2a\$; évite la double migration

Livrable du module

- ☒ `server/routes/auth.js` corrigé (bcrypt, messages génériques, rate limit, pas de log mdp)
- ☒ `scripts/migrate-passwords.js` créé et exécuté avec succès
- ☒ Base de données migrée : `SELECT password FROM users` retourne des hashes `$2b$12$ ...`
- ☒ Tests de validation passés : 6 commandes curl avec résultats documentés
- ☒ Section M2 ajoutée au rapport d'audit dans `docs/audit-report.md`
- ☒ Captures dans `proofs/M2/` (screenshots des tests curl ou output terminal)
- ☒ Commit avec tag `M2-auth-securise`

```
# Commit final
git add server/routes/auth.js scripts/migrate-passwords.js package.json
git commit -m "M2: correction auth, bcrypt + rate limit + messages génériques + migration mdp"
git tag M2-auth-securise

# Vérification du tag
git log --oneline -2
git tag
```

Section à ajouter dans docs/audit-report.md :

```
## Module M2, Authentification

### Failles corrigées

| ID | Faille | Sévérité | Statut |
|---|---|---|---|
| M2-1 | Mots de passe en clair (INSERT + comparaison) | Critique | ✓ Corrigé |
| M2-2 | Énumération de comptes (messages d'erreur différenciés) | Haute | ✓ Corrigé |
| M2-3 | Pas de rate limiting sur /login | Haute | ✓ Corrigé |
| M2-4 | Politique de mot de passe absente | Moyenne | ✓ Corrigé |
| M6-1 | Log du mot de passe en clair | Haute | ✓ Corrigé |

### Preuves techniques

- Hash bcrypt dans la DB : `SELECT password FROM users LIMIT 1` → `$2b$12$...`
- Rate limit actif : 6ème tentative de login → HTTP 429
- Messages uniformes : email inexistant et mauvais mdp → même réponse JSON
- Politique mdp : `POST /register` avec "court" → HTTP 400

### Risques résiduels M2

- Pas de 2FA : accepté (hors périmètre)
- Pas de vérification contre listes compromises (HaveIBeenPwned) : reporté
```

Erreurs fréquentes à éviter

- ⚠ **Oublier await avec bcrypt** : `bcrypt.hash()` et `bcrypt.compare()` retournent des Promises. Sans `await`, la route retourne `undefined` comme résultat de comparaison et tous les logins passent.
- ⚠ **Double migration** : exécuter le script deux fois sans vérification `startsWith('$2b$')` hash deux fois le mot de passe déjà hashé → login impossible.

- ⚠️ **Rate limit trop agressif** : 3 req/5 min bloque les utilisateurs légitimes qui tapent mal leur mot de passe. Calibrer selon le contexte.
- ⚠️ **Laisser** `const JWT_SECRET = "secret123"` dans le code : même si on le lit depuis `.env`, le fallback en dur est dangereux. En production, démarrer en erreur si la variable est absente.
- ⚠️ **Confondre** `bcrypt.hash()` et `bcrypt.hashSync()` : les versions synchrones bloquent l'évent loop Node.js. Toujours utiliser les versions async dans les routes Express.
- ⚠️ **Tester le rate limit depuis le même terminal sans reset** : après 5 tentatives, l'IP est bloquée 15 min. Pour re-tester, redémarrer le serveur (le store en mémoire se réinitialise).

Ressources

- [OWASP A02:2021, Cryptographic Failures](#)
- [OWASP A07:2021, Identification and Authentication Failures](#)
- [OWASP Password Storage Cheat Sheet](#)
- [OWASP Authentication Cheat Sheet](#)
- [NIST SP 800-63B, Digital Identity Guidelines](#)
- [bcryptjs npm](#)
- [express-rate-limit npm](#)
- [HaveIBeenPwned Passwords API](#)